

5. ADR-001. Стратегия эволюции архитектуры (от монолита к микросервисам).

Статус – предложено

Дата -

5.1 Контекст

Текущая архитектура системы размещения рекламы в формате аукционов в реальном времени AdScale представляет собой модульный (распределенный, гибридный монолит).

Система работает на пределе своих возможностей по производительности обрабатывает до 15 000 RPS с P95 latency 110–120 мс при целевом показателе в 100 мс. Ожидается рост нагрузки на 20% (до 18 000 RPS) к концу текущего квартала и двукратный рост (до 50 000 RPS) в течение года.

Архитектура характеризуется следующими критическими недостатками:

- **Узкое место и точка отказа:** Все компоненты (аукцион, статистика, финансы, аналитика) используют одну инстанцию БД PostgreSQL, что вызывает блокировку транзакций и деградацию времени отклика.
- **Отсутствие возможности масштабирования:** Отсутствие контейнеризации, очередей сообщений и кеширования делает невозможным горизонтальное масштабирование компонентов и всего сервиса .в целом
- **SLA нового партнера:** DSP-площадка требует стабильного ответа не более 80 мс и накладывает штрафы за простой.
- **Бизнес-ограничения:** Бюджет ограничен, срок промежуточного решения — 3 месяца, команда состоит из 35 человек без выделенных SRE. Полная переписывание системы исключено.

Необходимо выбрать стратегию трансформации, которая обеспечит работу под нагрузкой с увеличением на 20% (до 18 000 RPS) через 3 месяца и заложит фундамент для достижения 50 000 RPS через год.

5.2 Рассмотренные варианты

Вариант 1: Модульный монолит

Переход от распределенного монолита к модульному монолиту.

- Плюсы: Минимальные изменения инфраструктуры, сохранение производительности межмодульных вызовов (in-process).
- Минусы: Не решает проблему одновременного доступа к единственной БД. Не позволяет независимо масштабировать сервис статистики или аналитики. Сохраняет риск глобальных сбоев при деплое.

Вариант 2: «Big Bang» переписать все с нуля и переключить на новый сервис

Одновременное разделение монолита на десятки независимых микросервисов.

Не подходит вследствие бизнес-ограничений

Вариант 3: Паттерн Strangler Fig (Выбранный вариант)

Постепенное добавление к существующему коду монолита нового функционала в виде API Gateway и новых сервисов с последующей постепенной заменой устаревших частей.

- Плюсы: Обеспечивает непрерывность бизнеса. Позволяет начать с инфраструктурных изменений (очереди, кеш), дающих мгновенный прирост производительности. Снижает риски каждой итерации.
- Минусы: Временное существование распределенного монолита (гибридной архитектуры). Сложность поддержки двух путей обработки данных во время переходного периода.

Решение

Принять Вариант 3 (Strangler Fig Pattern) как основную стратегию миграции. Миграция разделена на два этапа

Этап 1. Период – 3 месяца

Повышение производительности инфраструктуры для возможности подключения DSP партнера. Вынос сервисов некритичных компонентов (аналитика)

Цель: Подключение DSP-партнера с сохранением требуемых SLA.

- **Внедрение асинхронной модели записи** в существующей архитектуре. Сервер сбора статистики отключаются от прямого доступа к БД. Создается брокер сообщений (кластер Apache Kafka). Запись событий осуществляется через запись в топик со временем отклика ~1–2 мс.

Далее запись в БД производится путем чтения из топика в фоновом режиме. Прямая синхронная запись в PostgreSQL устраняется.

- **Разделение транзакционных и аналитических потоков.** Разворачивается Read Replica PostgreSQL. SELECT-запросы сервиса аналитики и рекламного кабинета полностью перенаправляются на реплику.
- **Создание единой точки входа.** Создается реверсивный прокси перед внешними системами и пользователями, которая в последующем будет выполнять роль API Gateway (Envoy, Traefik, NGINX+, HAProxy) для управления маршрутизацией, аутентификацией и применения политик Circuit Breaker.

Альтернатива – выделение сервиса аукциона в отдельный микросервис на первом этапе и последующая репликация данного сервиса для возможности горизонтального масштабирования.

Достоинства – повышение производительности критичного сервиса

Недостаток – сложность решения, риск не выдержать срок. Не решает проблему одновременного доступа к единственной БД.

2 этап – реализация целевой архитектуры. Период – 1 год

Постепенный вынос модулей/сервисов в микросервисы согласно выбранному паттерну Strangler Fig

- **Изоляция сервисов статистики и аналитики.** Создаются независимые микросервисы статистики и аналитики с собственными БД (или репликой БД). Монолит выступает исключительно в роли источника данных (событий) в Kafka.
- **Повышение времени отклика для чтения из БД.** Внедряется кеширование наиболее часто читаемых данных в Redis Cluster (например, данные пользователей) для исключения блокировок PostgreSQL БД
- **Вынос бизнес-логики в отдельный высокопроизводительные микросервисы (C++/Go),** доступ к которому осуществляется по gRPC. Это повышает производительность и снижает сложность системы в целом.
- **Управление данными.** Создание выделенных БД для новых микросервисов. Запрет на прямые SQL-обращения новых сервисов к таблицам старого монолита. Синхронизация данных новых БД с мастер данными в существующей БД через механизм Change Data Capture . Рассмотреть выбор БД для сервиса аналитики в отдельном ADR – предпочтительный тип БД для данного сервиса, ClickHouse для оптимизации OLAP-нагрузки.

- **Изоляция финансовых других операций.** Финансовые расчёты также становятся асинхронным и исключаются из критического пути ответа на ставку. Также выносятся в микросервисы оставшиеся модули/сервисы.

3. Этап. После реализации целевой архитектуры и по мере роста нагрузки

Горизонтальное масштабирование и финальная миграция

- **Шардирование** наиболее нагруженных таблиц для БД аукциона по мере роста нагрузки.
- **Повышение количества реплик** наиболее нагруженных сервисов по мере роста нагрузки и балансировка на них через API Gateway
- **Мультирегиональное развертывание.** Архитектура разворачивается в нескольких зонах доступности. Глобальный балансировщик DNS направляет трафик в ближайший регион, снижая сетевую задержку (RTT).

Последствия решения

Положительные

- Снижение рисков: Постепенный подход, начиная с наименее критичного, но наиболее ресурсоемкого сервиса позволяет откатить любой выделенный сервис без остановки всей платформы.
- Изоляция ресурсов: Проблемы в аналитике или статистике (тяжёлые SQL) больше не влияют на критический RTB-путь.
- Новые интеграции (DSP) могут подключаться через фасад, используя очереди, не дожидаясь полной очистки легаси-кода.
- Горизонтальная масштабируемость: Возможность добавлять вычислительные мощности путем простого увеличения количества реплик и шардирования БД.
- Устранение единой точки отказа: Выход из строя сервиса статистики больше не влияет на способность системы отвечать на ставки.
- Ускорение разработки: команды разработки могут владеть отдельными сервисами целиком,

Отрицательные / Риски

- Сложность экспликации решения в переходный период: команде придется поддерживать как микросервисы так и монолит одновременно.
- Существует риск рассинхронизации данных между старой схемой монолита выделенными БД для микросервисов и новыми сервисами.
- Повышение сетевой задержки (Network Latency): Внутрисервисное взаимодействие (gRPC/HTTP) добавляет 1–3 мс.

- Усложняется end-to-end тестирование сквозных процессов (например, от клика до списания средств). Трассировка запроса, проходящего через 4–5 сервисов, требует внедрения распределённой трассировки